Two builds of the Northstar IT Help Desk

Venkata Naga Kiran Katari Banner ID 916503744

I built the same help desk twice. The first version is one Python file, written while the program was running. The second version started as six specification documents, and the code was written only after those documents were reviewed. Both versions answer from local IT documents, check a simulated service status, and wait for the employee to confirm a ticket before SQLite stores it.

Side by side

	One-file build	Specification-first build
Code	demo1_vibe/helpdesk_app.py	demo2_spec/, separate modules
Written first	The running program	Six documents in specs/
Automated tests	6 passed	34 passed
Ticket shown	TCK-1002, urgent, open	TCK-1001, urgent, open
Evaluation	4 of 4 passed, 90 ms	14 scripted cases

One-file build

I put the chat, the TF-IDF search, the status check, the ticket draft, and the confirmation step in a single Streamlit program. The sample question is an employee whose VPN drops before a client meeting. The reply cites the Northstar VPN document, tells the employee to move the GlobalProtect gateway to US-Central, and asks which device they are using. The ticket stays a draft until Confirm is clicked.

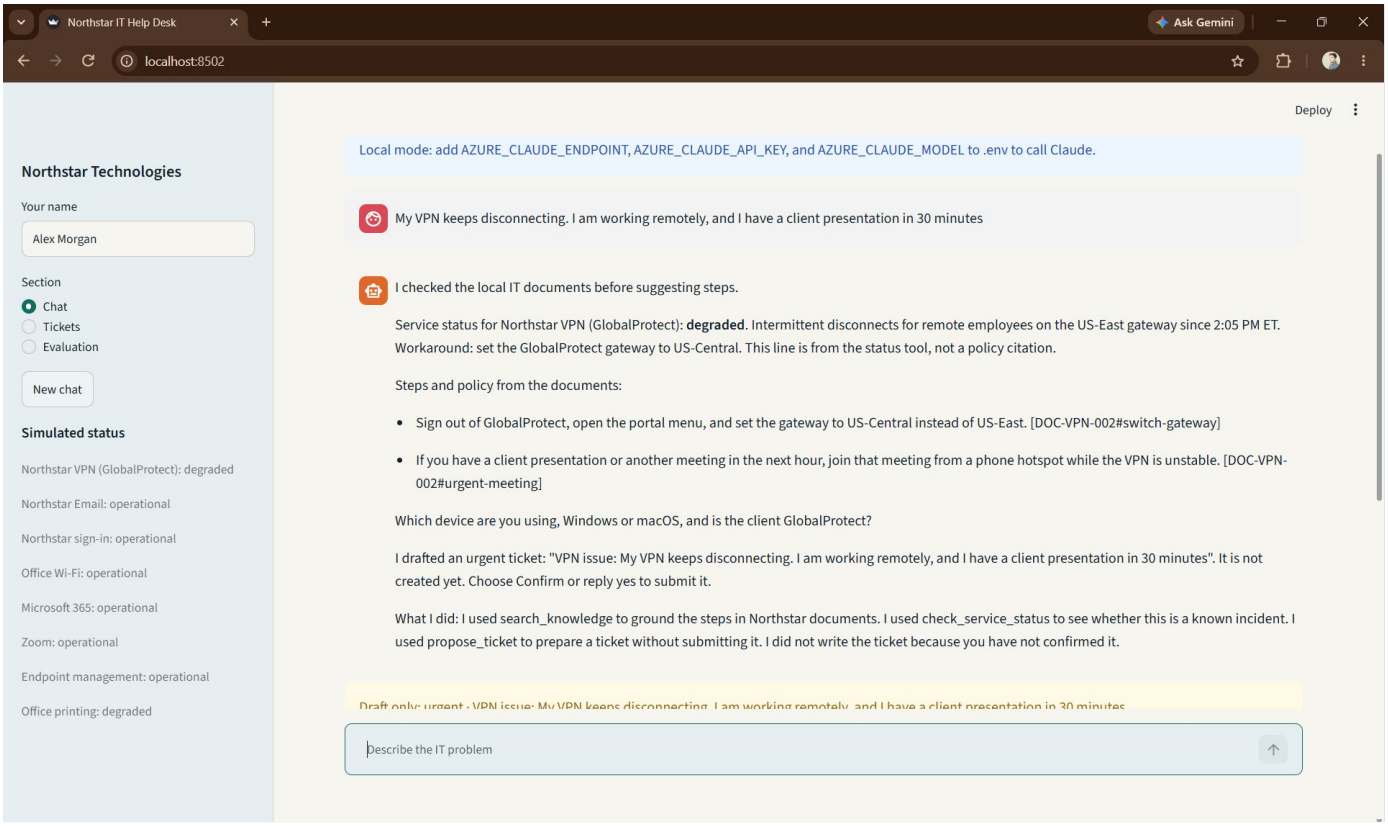


Figure 1. The one-file build cites DOC-VPN-002 and says the urgent ticket has not been created.

The trace lists the tools that ran. It does not print private model reasoning. A message that tries to override the instructions is refused before any tool runs. Azure Claude is read from a local .env file, and no key is stored in the repository. The demo and the tests use the local answer path. pytest collected 6 tests for this build, and all 6 passed.

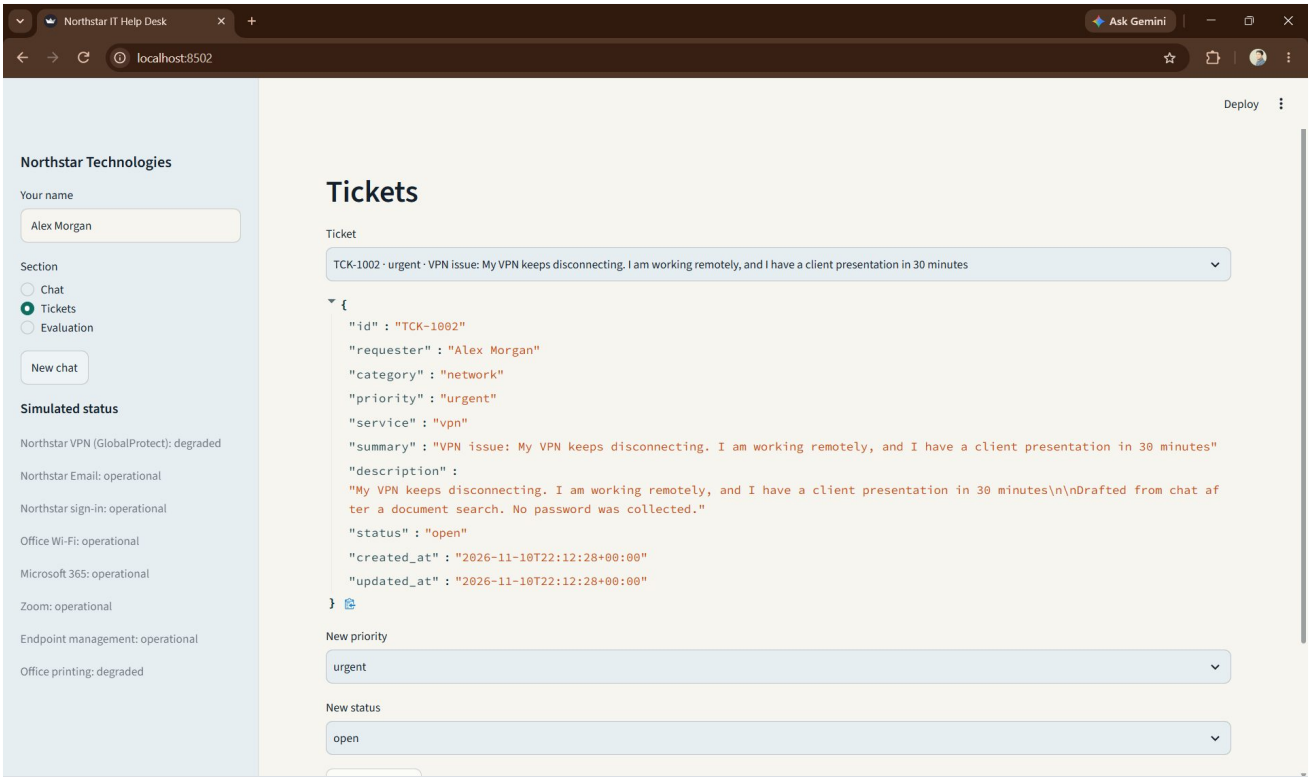


Figure 2. TCK-1002 is stored only after confirmation. Priority is urgent and status is open.

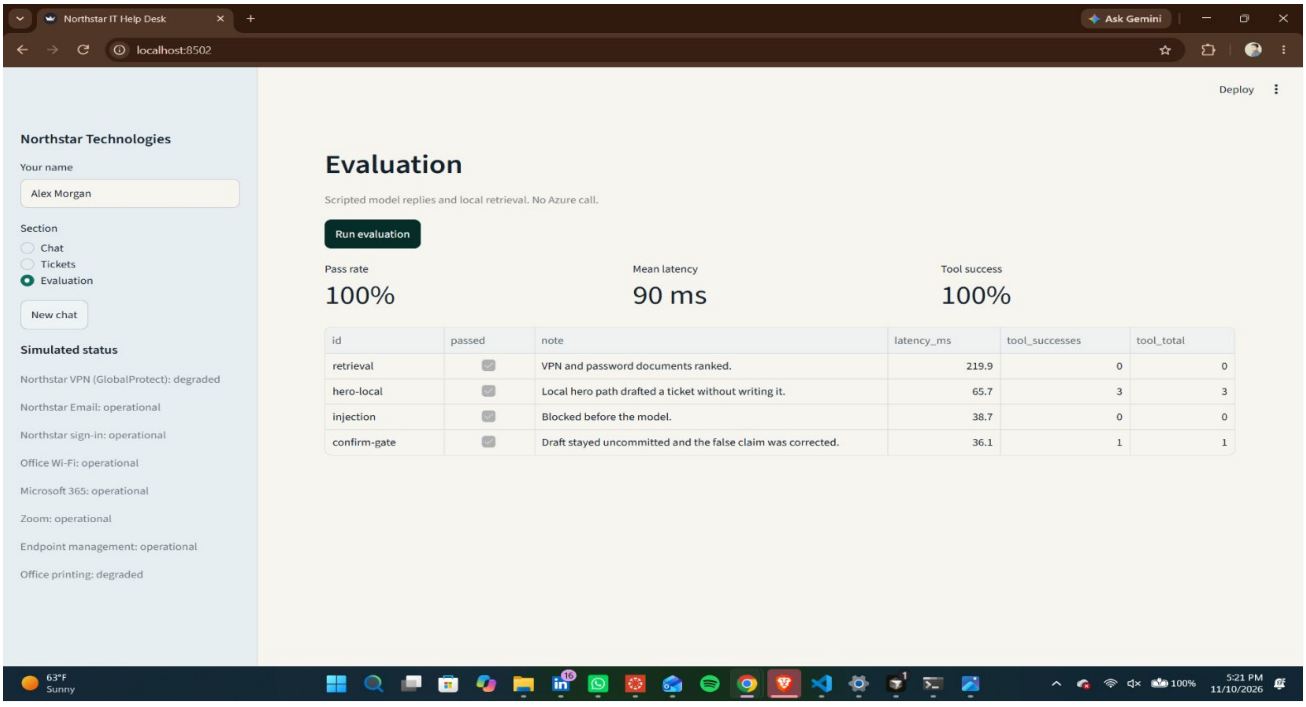


Figure 3. The one-file evaluation passes 4 of 4 cases, with a mean latency of 90 ms and 100 percent tool success.

Specification-first build

The second build does not import the first program. I wrote requirements.md, architecture.md, agent-design.md, security.md, evaluation.md, and implementation-plan.md before the application code. Each feature has an identifier, the expected behavior, acceptance criteria, test cases, and error handling.

I then checked those documents for gaps and for steps that should stay ordinary Python. Citations have to match a retrieved passage. An unknown tool is rejected. A ticket write waits for an explicit confirmation. specs/review.md maps each requirement to a test. The model client does not decide when a ticket may be saved, and the automated tests use a mock model, so they do not call Azure.

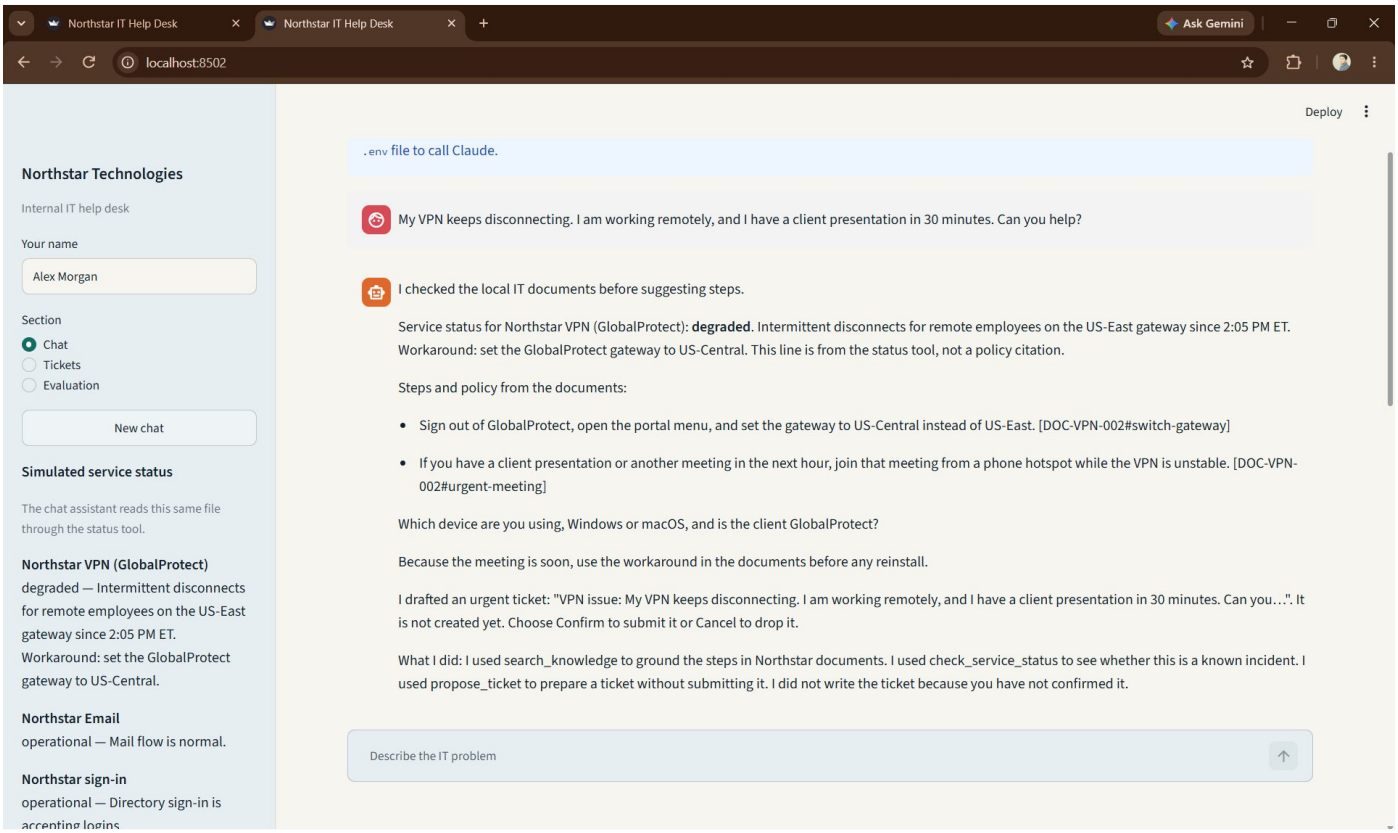


Figure 4. The specification-first build shows the US-Central step, the document citations, and a draft that is still outside the database.

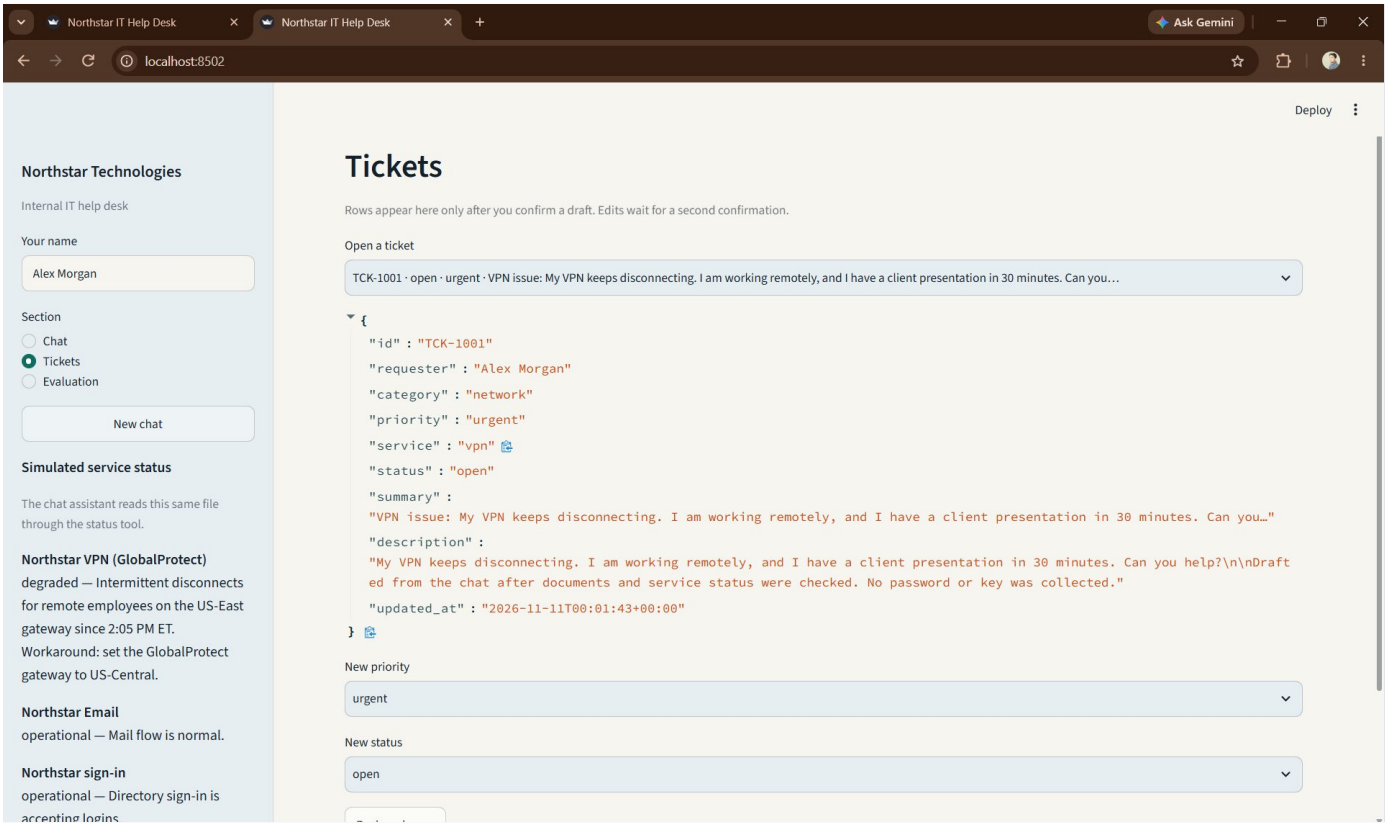


Figure 5. TCK-1001 appears on the Tickets page only after the employee confirms the draft.

What the specification changed

pytest collected 34 tests for this build, and all 34 passed. The evaluation page runs the same 14 scripted cases used by those tests and reports pass rate, mean latency, and tool success. Because each case points back to a requirement, a failing test names the rule it broke.

The one-file build was the faster way to get a working demo. The specification-first build is the one I would keep working on, because the documents, the modules, and the tests describe the same behavior.